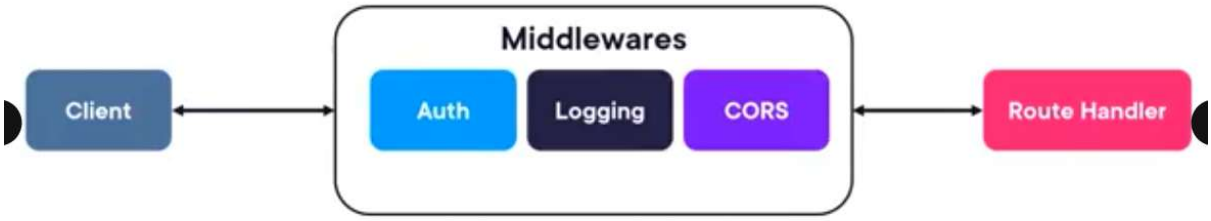
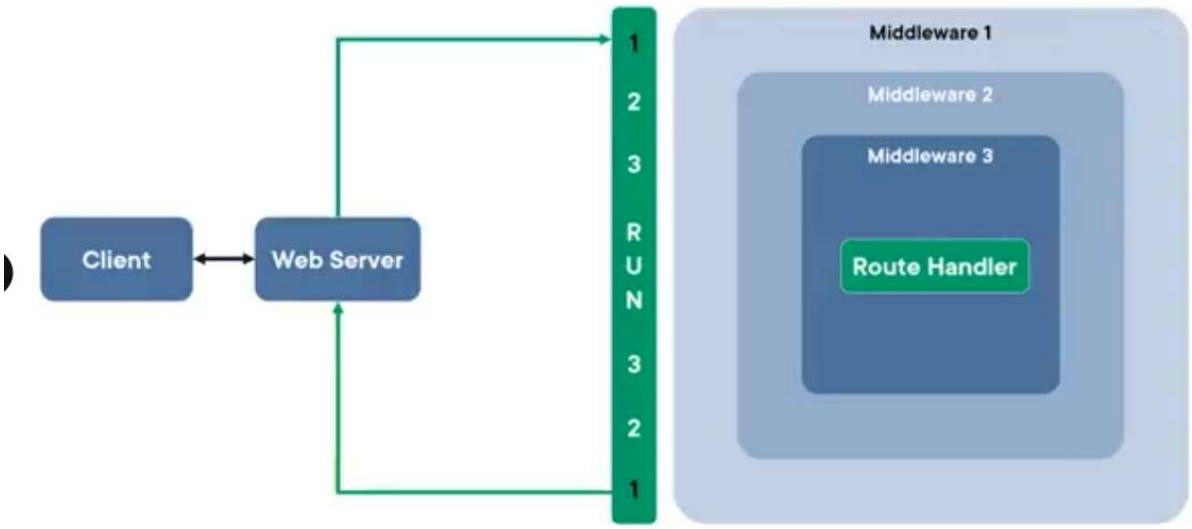
4. FASTAPI ADVANCED REQUEST HANDLING AND MIDDLEWARE

4.1 Understanding Fast API middleware architecture



This diagram illustrates the request-response cycle in a web application backend, specifically highlighting the role of middleware between the client and the final route handler.

- **Client**: The initiator of the HTTP request, such as a web browser or mobile app.
- **Middlewares**: Intermediary functions that process the request before it reaches the handler, including **Auth** (authentication), **Logging** (recording activity), and **CORS** (cross-origin resource sharing).
- **Route Handler**: The final destination that processes the request and sends the response back to the client.



The image displays a diagram representing a web application architecture, illustrating how a request flows from a client through a web server and a middleware pipeline to a route handler, and back.

- **Client & Web Server**: The client initiates a request, which is received and processed by the web server.
- **Middleware Pipeline**: The request passes through nested middleware layers (Middleware 1 -> Middleware 2 -> Middleware 3) to handle cross-cutting concerns like logging or authentication before reaching the core application logic.
- **Route Handler**: The request is finally processed by the route handler, which generates the appropriate response.
- **Return Flow**: The response travels back through the middleware layers in reverse order (Middleware 3 -> Middleware 2 -> Middleware 1) before returning to the web server and finally the client.

Each middleware is a powerful structure as each each controls the response to be generated .

ASGI : stands for – Asynchronous Server gateway Interface

- ➔ ASGI is the protocol that enables communication between your web server and your FastAPI application
- ➔ Fast API builds Starlette , which provides the ASGI implementations.

Middleware common usecases :

1. Authentication
2. Logging
3. Monitoring
4. CORS
5. Error handling

Throughout the course you will be implementing , Middleware with Inventory Management API

Using middleware to add Authentication , authorisation and etc things in project created in fastapi 3.

Demo :

Steps:

1. Start by creating venv environment
2. Activate the environment
3. Then install requirements.txt file
4. Setup the database
5. Instailizing database by using management/inventory.py file
6. Run api : `python -m app.main`

Creating middleware:

1. In main directory App create a new folder and name it as middleware
2. In the folder create a new file and name it as logging as it used for logging data
3. In this file we will be creating timing middleware to see see how long it takes the request to finish .
4. Start by importing all the modules : where BaseHTTPMiddleware is the base middleware case of http used for creating middlewares

```
import time
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
```

5. Creating a class for creating logging middleware: important thing here is the dispatch that is to be called whenever a request is made

```
class LoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
```

6. Then there is a variable called response that call call_next and this is the actual route handle that handles request and returns data

```
response = await call_next(request)
```

7. Defining variable to calculate the start time

```
start_time = time.perf_counter_ns()
```

8. A variable to calculate process time in milli seconds and round it up

```
process_time_ms = round((time.perf_counter_ns() - start_time) / 1_000_000, 2)
```

9. Return the process timed in millisecond and in string format

```
response.headers["X-Process-Time-Ms"] = str(process_time_ms)

return response
```

10. Registering the middleware in main.py file

11. First we need to import it

```
from app.middlewares.logging import LoggingMiddleware
app.add_middleware(LoggingMiddleware)
```

12. save the changes and run the program to get the response with processing time in milliseconds

Response headers

```
content-length: 333
content-type: application/json
date: Tue,22 Apr 2025 16:49:37 GMT
server: uvicorn
x-process-time-ms: 50.49
```

demo-2 :

Extending the logging middleware to capture logging details :

Steps:

1. starts by defining a request log data class as the reuest model class for defining the format of taking request input format

```
class RequestLogData(BaseModel):
    timestamp: datetime
    request_id: str
    method: str
    path: str
    status_code: int
    duration_ms: float
    client_ip: Optional[str] = None
    user_agent: Optional[str] = None
```

2. now making changes in the main LoggingMiddleware class
3. lets start by creating a new varaibel that helps us define a universal unique id and also import uuid .

```
request_id = str(uuid.uuid4())
```

4. and then define and create log data and define the new instance of log that is to be created

```
log_data = RequestLogData(
    timestamp=datetime.now(),
    request_id=request_id,
    method=request.method,
    path=request.url.path,
    status_code=response.status_code,
    duration_ms=process_time_ms,
    client_ip=request.client.host if request.client else None,
    user_agent=request.headers.get("user-agent", "")
)
```

5. and adding another header for request id :

```
response.headers["X-Request-ID"] = request_id
```

6. defining a class default_log_handler that is going to log the output this is an log handler and needs to import logging and system

```
def default_log_handler(log_data: RequestLogData):
    logger = logging.getLogger("api_logger")
    if not logger.handlers:
        logger.setLevel(logging.INFO)
        handler = logging.StreamHandler(sys.stdout)
        formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
        handler.setFormatter(formatter)
        logger.addHandler(handler)

    logger.info(
        f"{log_data.method} {log_data.path} - "
        f"ID: {log_data.request_id} - "
        f"Status: {log_data.status_code} - "
        f"IP: {log_data.client_ip} - "
        f"UA: {log_data.user_agent} - "
        f"Duration: {log_data.duration_ms}ms"
    )
```

7. then calling the function created in middleware class .

```
default_log_handler(log_data)
```

we create a logging middleware that was logging information but there was a problem but is dynamic data that means database and this implementation is not practical

and this is demo is about how to configure the middleware to add configuration from outside and do file logging .

steps:

1. in core folder create a new file called logging.py
2. creating a class logger factory so that whenever we have to change anything we just need to change the logger data
3. creating a function create file logger inside the class , this function takes data like name , level , file_path a, max_bytes and backup_count , after this it is going to create a new file after this.

```
@staticmethod
def create_file_logger(
    name="file_api_logger",
    level=logging.INFO,
    file_path="api_requests.log",
    max_bytes=10*1024*1024,  # 10MB
    backup_count=5
):
```

4. using file handler for handling file instead of getting an output in the console and get_logger is helping us log messages

```
logger = logging.getLogger(name)
if not logger.handlers:
    logger.setLevel(level)
    file_handler = RotatingFileHandler(
        file_path,
        maxBytes=max_bytes,
        backupCount=backup_count
    )
```

   The provided image displays a Python code snippet that sets up a logger to write to a rotating file.
   a. It creates a logger instance using logging.getLogger(name).
   b. It checks if the logger already has handlers to prevent duplicate logging.
   c. It configures a RotatingFileHandler to manage log files, allowing for file rotation based on size (maxBytes) and keeping a specific number of old log files (backupCount).
5. In middleware class we will be define a constructor class so we can pass data while we intialize middleware

```
def __init__(
    self,
    app,
    log_handler: Optional[Callable[[RequestLogData], None]] = None ):
```

This code snippet defines the constructor (__init__) method for a class, likely used to initialize an application component.

a. It accepts a self parameter (instance reference) and an app parameter.
b. It includes an optional parameter log_handler, which is a type-hinted function that takes RequestLogData and returns None.
c. The log_handler defaults to None if not provided.

6. Make sure to intialise the http middleware as well

```
super().__init__(app)
self.log_handler = log_handler or default_log_handler
```

The provided code snippet demonstrates how to initialize a parent class and set up a logging mechanism in Python.

a. super().__init__(app) calls the initializer of the parent class, passing app as an argument.
b. self.log_handler = log_handler or default_log_handler assigns a log handler to the instance, utilizing a default if the specified log_handler is not provided.
c. Not writing the statements are not going shows us any error but writing __init__(app)then it will fail

7. Instead of doing log handler we will be using self.log_handler instead
8.

```
self.log_handler(log_data)
```

function: This code likely calls a function named log_handler belonging to the current instance (self), passing log_data as an argument to be processed or recorded.

9. Next step is to configure main file and providing info about log handler and for this we will be using the logger factory that was being created
10. In main.py import and include loggerfactory class

```
from app.core.logging import LoggerFactory
```

```
console_logger = LoggerFactory.create_console_logger()
file_logger = LoggerFactory.create_file_logger(file_path="logs/api_requests.log")
```

11. As we have loggers now we will not pass logger loggers directly into the middleware as it excepts a log_handler and this function we can also define it in logger file of core module

```python
def custom_log_handler(log_data: RequestLogData):
    log_message = (
        f"{log_data.method} {log_data.path} - "
        f"ID: {log_data.request_id} - "
        f"Status: {log_data.status_code} - "
        f"IP: {log_data.client_ip} - "
        f"UA: {log_data.user_agent} - "
        f"Duration: {log_data.duration_ms}ms"
    )
    console_logger.info(log_message)
    file_logger.info(log_message)
```

The provided code snippet defines a Python function named custom_log_handler that formats request log data into a structured string and logs it to both the console and a file.

a. **Function Purpose:** Formats log_data into a single, comprehensive string containing key request information.
b. **Log Contents:** Includes HTTP method, path, request ID, status code, client IP, user agent, and duration in milliseconds.
c. **Logging Targets:** The formatted message is sent to both console_logger and file_logger.

12. Add this method in the middleware

```python
app.add_middleware(LoggingMiddleware,log_handler=custom_log_handler)
```

13. This is going to create a api_request.log file where all the request log are to be inserted .

4.2 Building Custom Authentication and Authorization Middleware

Currently our fastapi application does not have any authentication that means anybody can call the api

In this module you will be coding to add middleware to secure requests between a client and the api sender

The first step is to get an endpoint where user can user to get authentication for api

We already have the endpoint present in code file the post takes email and password as input and returns jwt token

We can verify the token by passing the generated token in jwt.io website

In the code file we cannot add authorization header as api is not configured to do so

Steps:

1. Update the swagger configuration to take up authentication and this is to be done by adding a dependencies
2. Under app create a new folder called dependencies
3. Importing security from fastapi

```
from fastapi import Security
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
```

   The provided code snippet demonstrates how to import security components from the FastAPI framework to implement token-based authentication.
   a. Security: A dependency injection utility used to define security requirements for endpoints.
   b. HTTPBearer: A dependency class that automatically handles the extraction and validation of a Bearer token from the Authorization header.
   c. HTTPAuthorizationCredentials: A class representing the credentials extracted by HTTPBearer, typically containing the token string.
4. Specifying a bearer_schema that is an HTTPBearer

```
bearer_scheme = HTTPBearer(auto_error=False)
```

   This code snippet defines a security scheme for handling bearer token authentication in a web application framework, likely FastAPI.
   a. bearer_scheme: The variable name assigned to the security mechanism.
   b. HTTPBearer: The class used to define the authentication type as a bearer token.
   c. auto_error=False: Configures the scheme not to automatically raise an error if authentication fails, allowing the code to handle the missing token manually.
5. Defining an function which is dependency function that does not have any functionalities but is telling swagger to display a lock symbol

```
def ensure_bearer(credentials: HTTPAuthorizationCredentials = Security(bearer_scheme)):
    # Do nothing - this is just to make Swagger show the 🔒 button
    return
```

6. Register this is main.py file

```
from app.dependencies.auth import ensure_bearer
```

```
pp = FastAPI(
    title="Inventory Management API",
    description="API for managing inventory across multiple locations",
    version="1.0.0",
    dependencies=[Depends(ensure_bearer)]
```

Demo:3

In this demo we are going to create a middleware which is going to parse an authorization header for request and then validate the token and parse the user details into our request.

Steps:

1. In middleware folder create a new file authentication.py
2. Creating class and importing the import modules and function

```
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse
from typing import Optional
from pydantic import BaseModel
from app.core.security import decode_jwt_token

class AuthenticatedUser(BaseModel):
    id: str
    email: str
    roles: list[str]
    first_name: Optional[str] = None
    last_name: Optional[str] = None
```

This image displays Python code defining a data model for an authenticated user within a web application framework.

a. It imports necessary modules for middleware, HTTP requests, JSON responses, and Pydantic for data validation.
b. A Pydantic BaseModel class named AuthenticatedUser is defined to structure user data.

c. The model includes fields for user ID, email, roles, first name, and last name, with optional fields handled using Optional.

3. Creating jwt authentication middleware
4. Start by creating a middleware class

```python
class JWTAuthenticationMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        request.state.user = None

        auth_header: Optional[str] = request.headers.get("Authorization")
        if auth_header and auth_header.startswith("Bearer "):
            token = auth_header.removeprefix("Bearer ").strip()

            try:
                claims = decode_jwt_token(token)

                request.state.user = AuthenticatedUser(
                    id=claims["sub"],
                    email=claims["email"],
                    roles=claims.get("roles", []),
```

This code snippet defines a Python middleware class, JWTAuthenticationMiddleware, for handling JSON Web Token (JWT) authentication in a web application. It extracts a token from the Authorization header, decodes it, and sets the authenticated user in the request state.

a. **Middleware Class**: JWTAuthenticationMiddleware processes incoming HTTP requests to check for user authentication.
b. **Token Extraction**: It looks for an Authorization header starting with "Bearer " and extracts the token string.
c. **JWT Decoding**: The token is decoded using a decode_jwt_token function to extract user claims.
d. **User Assignment**: Extracted claims like user ID, email, and roles are used to populate request.state.user.

```python
try:
    claims = decode_jwt_token(token)

    request.state.user = AuthenticatedUser(
        id=claims["sub"],
        email=claims["email"],
        roles=claims.get("roles", []),
        first_name=claims.get("first_name"),
        last_name=claims.get("last_name"),
    )
    print(request.state.user)
except ValueError:
    return JSONResponse(
        {"detail": "Invalid or expired token"},
        status_code=401
    )
```

This code snippet demonstrates a Python function to decode and validate a JWT (JSON Web Token) within a request handling process, typically used for user authentication in web APIs.

- **Token Decoding**: It attempts to decode a provided JWT using a decode_jwt_token function.
- **User Authentication**: Upon successful decoding, it populates a request.state.user object with user details extracted from the token's claims, such as id, email, roles, and name.
- **Error Handling**: If a ValueError occurs during decoding (e.g., invalid signature, expired token), it returns a 401 Unauthorized JSON response indicating the token is invalid or expired.

5. Let’s register the middleware in main.py file

```python
from app.middlewares.authentication import JWTAuthenticationMiddleware
app.add_middleware(JWTAuthenticationMiddleware)
```

Until this step With our current json web authorization header we can still call API without an authorization as we can try using product endpoint and it will still work

Now creating the second part of the demo where we need to make sure that certain route

This is done by dependencies :

Dependencies :fastAPI dependies are reusable functions that encapsulate logic and can be injected automatically into endpoints at the app , router or route level

Steps:

1. Navigating into auth.py file of dependencies folder
2. Creating a new function that check if the user is authenticated

```
def get_current_user():
    def dependency(request: Request):
        user = getattr(request.state, "user", None)
        if not user:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Authentication required"
            )
    return dependency
```

   This code defines a dependency function for a web framework (likely FastAPI) that checks if a user is authenticated.
   - It attempts to retrieve a user object from the request's state.
   - If no user is found, it raises an HTTP 401 Unauthorized exception.
   - Otherwise, it returns the dependency function, allowing the request to proceed.
3. Register the dependecencies
4. Go to router and use and place in router where ever we authentication
5. Example : in router.product.py file
6. We will import first

```
from app.dependencies.auth import get_current_user
```

7. All these routes inside router will need authorization so we will define it as dependency in router

```
10  router = APIRouter(
11      prefix="/products",
12      tags=["Products"],
13      dependencies=[Depends(get_current_user())]
14  )
```

This code snippet is written in **Python** using the **FastAPI** framework. Specifically, it shows how to apply global security or logic to a group of routes.

- **What is happening in this code?**
- The highlighted line dependencies=[Depends(get_current_user())] is a powerful FastAPI feature used for **Dependency Injection**.
- **router = APIRouter(...)**: This creates a "mini-application" (a router) specifically for products. Instead of putting all your code in one file, you use routers to organize your API into logical sections (e.g., users, products, orders).
- **prefix="/products"**: Every endpoint inside this router will automatically start with /products (for example, GET /products/123).
- **dependencies=[Depends(...)]**: This is the "gatekeeper." By adding this at the router level, you are telling FastAPI: **"Before running any code in any route within this router, run the get_current_user function first."**
- **Why is this used?**
- **Authentication/Authorization:** This is most commonly used to ensure that a user is logged in before they can access any product-related data. If get_current_user fails (e.g., the token is invalid), the request stops there and returns an error (like 401 Unauthorized).
- **Code Cleanliness (DRY):** Instead of manually checking for a user inside every single product function, you define it once at the top.
- **Scalability:** It makes it easy to apply the same security rules to an entire category of features at once.
- **Summary of the variables:**

Component	Meaning
prefix	The URL path prefix for all

	routes in this group.
▪ **tags**	▪ Used for grouping these routes in the auto-generated Swagger UI documentation.
▪ **dependencies**	▪ A list of functions that must execute successfully before the route logic runs.
▪ **Depends**	▪ A FastAPI class that tells the framework to treat the function inside as a dependency.

8. That means whenever any route is called this dependency is to be run first

Our database in our inventory management API contains the following users : as we have super admin, inventory manager and sales clerk and all of them have different role

Name	Email	Roles
Super Admin	admin@globomantics.com	admin
Inventory Manager	inventory@globomantics.com	inventory_manager
Sales Clerk	clerk@globomantics.com	sales_clerk

For example: sales clerk is not allowed to update inventory but inventory manager and UER admin should be allowed to do so

Here we will be adding role-based authorization and now adding check against roles of the user

Updating get_current_user to authorize on the basis of roles

```
def get_current_user(required_roles: Optional[List[str]] = None):
    def dependency(request: Request):
        user = getattr(request.state, "user", None)
        if not user:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Authentication required"
            )

        if required_roles:
            user_roles = getattr(user, "roles", [])
            if not any(role in user_roles for role in required_roles):
                raise HTTPException(
                    status_code=status.HTTP_403_FORBIDDEN,
                    detail="You do not have access to this resource"
                )

        return user
    return dependency
```

2. go to inventory database pattern and make some changes

It means authentication happens early in the request lifecycle and every route can then simple depends on a function to access the user and enfornce permissions .

By combining theses with fastapi dependency injection system we get a clean reusable logic for role based access control